

Final Project - EECS 211

Winter 2026

1 Overview

In this final project, you will modify a tick-based kernel to dynamically adjust the tick period under the kernel's control. Your goal is to modify the kernel so that the tick period will be dynamic. You should have a policy for how the tick period changes based on the behavior of the processes. You are required to measure the difference in the performance and write a report about your adaptive tick interval algorithm.

Lab environment: All tasks in this lab are performed on EECS-provided Linux servers.

Readings and references: Students are encouraged to consult the following resources for additional background:

- Xv6, a simple Unix-like teaching operating system

Requirements: The objective of this project is to implement a dynamic tick interval policy. This should respond to the operation of processes in the system with the goal of reducing the number of ticks while preventing a process from monopolizing the CPU.

You should use the runtime program provided in the user folder to test the performance of your dynamic tick interval policy compared with the original fixed interval algorithm. When evaluating your policy:

- Use `forktest`, `usertests`, and `ls` programs.
- Run an average of 10 times each.
- Report the performance comparison in your report. Metrics to include: execution time, total number of ticks, context switches, etc.

You will upload one zip file that contains:

- **PDF report**, which may include:
 - A description of your dynamic tick policy
 - What system information your policy uses
 - Performance comparison between fixed and dynamic tick kernels
 - Screenshots of terminal output
 - A discussion of trade-offs or limitations
- The code template folder (`xv6-riscv` folder) after finishing your implementation.

Note: Points will be deducted if no screenshots or sufficient explanations are given. The students should also **fill the honesty statement and upload it to canvas**, otherwise 30 points will be deducted.

Due date: 03/15/2026, 23:59:59 PST

2 Background

For this assignment, you will use the xv6 kernel (provided as a zip file named `xv6-riscv-riscv.zip`) for the RISC-V 64 bit architecture. This includes all of the core components of an operating system kernel while still being accessible to hack on.

3 xv6 Kernel

The xv6 kernel includes the various components you would expect to be in an operating system kernel.

- **Scheduler:** The actual scheduler is in `proc.c`. You should be able to see that the scheduler by default uses a round-robin scheduler.
- **Interrupts/Exceptions:** The code for handling interrupts and exceptions is in `trap.c`. This is where the code resides that handles when a userspace process calls a syscall (trapping to the kernel) and where interrupts are handled.
- **Syscalls:** Userspace applications have a very standard set of syscalls, including `read`, `write`, `fork`, etc.

4 Setup

1. Extract the xv6 Kernel zip file.
2. Test the installation:

```
cd xv6-riscv-riscv
make qemu
```

This will load the emulator with the kernel. By default, the kernel loads the shell process (`sh`). This is a basic shell, but should be reasonably familiar.

3. You can use this basic shell to execute all the existing applications in the user folder.
4. To exit QEMU, you can enter `ctrl+a` and then `x`.

5 How to modify the kernel to implement a dynamic tick interval policy

To modify the scheduler to support a dynamic tick interval policy, you can directly edit the scheduler function inside `proc.c` and then re-run `make qemu`.

You can use the behavior of processes and CPU to set a time interval for the current process based on your chosen dynamic tick interval algorithm. You can find some useful variables defined in `cpu` and `process` structs (`proc.h`), such as the total number of running processes, the total number of sleeping processes, etc.